Algorithmic Robotics
COMP/ELEC/MECH 450/550
Project 3 Part 1: Random Tree Planner

Darshon Singh (ds161) and Winson Lin (wl85)

Problem

We implemented the Random Tree Planner (RTP) in C++ using OMPL and planned motions for the seven-joint Panda arm from Project 2. Two custom 3D scenes contain sphere and oriented-box obstacles. In each query, the start and goal are valid but direct joint-space interpolation collides, so RTP must find an alternative route.

Panda arm and its joint space

The Panda has seven revolute joints. The base and gripper are fixed, so a configuration is just the seven joint angles

$$q = (q_1, \ldots, q_7), \qquad \mathcal{C} = \prod_{i=1}^{7} [q_i^{\min}, q_i^{\max}] \subset \mathbb{R}^7$$

with these limits from the URDF (radians):

joint	1	2	3	4	5	6	7
min	-2.97	-1.83	-2.97	-3.14	-2.97	-0.09	-2.97
max	2.97	1.83	2.97	0.09	2.97	3.82	2.97

OMPL represents this box as `RealVectorStateSpace(7)` and interpolates linearly between joint vectors. Forward kinematics transforms 59 link-attached collision spheres into the world frame for checks against environment spheres and oriented boxes. Self-collision and velocity or acceleration limits are outside the assignment's model. The joint-box diagonal is $D \approx 13.4165$ rad, so resolution 0.005 corresponds to a nominal maximum interpolation segment length $0.005D \approx 0.0671$ rad. OMPL chooses an integer number of segments for each edge; actual spacing depends on the edge length and segment count.

RTP implementation

Each tree vertex stores an OMPL state and a parent pointer in a `std::vector`. Each iteration selects an existing vertex uniformly, samples a target uniformly or from the goal with probability equal to the goal bias, and calls `checkMotion()` on the full edge. Only an accepted target is cloned and attached. There is no nearest-neighbor search or range limit. Goal satisfaction is checked for existing and new vertices; parent pointers recover the path back to the start, then reverse it.

While planning we keep track of the vertex with the smallest goal distance. If time runs out, we return the path to that vertex as an approximate solution. The planner owns every state and `Motion` it allocates, and `freeMemory()` releases all of them. Both `clear()` and the destructor call it. Solution paths hold independent state copies. `getPlannerData()` exports the tree vertices and parent-to-child edges, labels roots as starts, and labels an exact solution endpoint as a goal.

We reviewed the implementation and rebuilt it with OMPL 2.0.2 on macOS and Ubuntu Linux (ARM64). A separate lifecycle check passed 25 clear/reuse cycles, exact and approximate solutions, continued solving, a start already at the goal, invalid-start rejection, and planner-data labels and edges. Counting state allocations and frees found no remaining allocated states after the test objects were destroyed. These checks exercise RTP directly rather than only the supplied arm interface.

For the four saved custom-scene paths, we checked the embedded scenes against the current YAML files, the start and goal, joint limits, and the parent relationships in the exported trees. An independent forward-kinematics calculation matched the supplied Pinocchio fixture within 10^{-10} m. All four saved paths passed collision sampling with joint-space intervals at most 0.002 rad. This is a denser sampled check, not a proof of continuous collision freedom. The assignment's planning resolution remained 0.005.

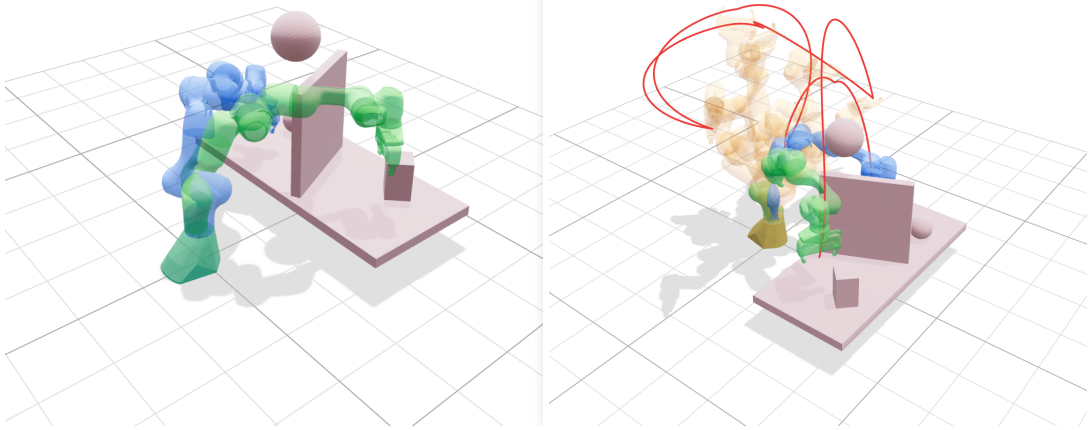
Custom scenes

Both scene files, including the exact seven-angle start and goal vectors, are in `resources/scenes/`. Positions, sphere radii and box half-extents are in meters; obstacle quaternions use $[x, y, z, w]$ order. The approximate hand positions below are centroids of the hand collision spheres. In all figures the blue arm is the start, the green arm is the goal, the faded orange arms are the intermediate waypoints of the RTP path, and the red curve is the path of the gripper.

Wall (`wall.yaml`). a table in front of the robot with a thin wall standing on it, rotated about 25 degrees around z, which splits the table into a left ($+y$) side and a right ($-y$) side. A ball at height 0.78 m obstructs space above the wall; a smaller ball and a rotated block add clutter. Both queries start with the hand pointing down on the left side of the wall and end on the right side. In query 1 the hand goes from $(0.46, 0.37, 0.27)$ to $(0.62, -0.29, 0.39)$ m and in query 2 from $(0.54, 0.42, 0.39)$ to $(0.51, -0.23, 0.52)$ m. Direct interpolation intersects the wall in both queries; query 2 also intersects the hanging ball.

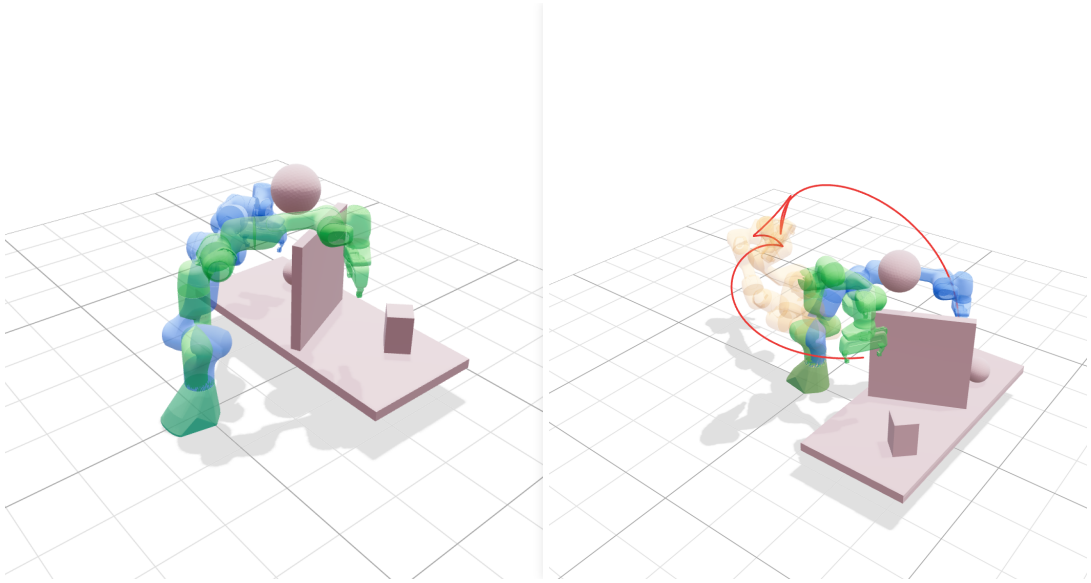
Window (`window.yaml`). a window frame in the plane $x = 0.45$ m with an opening about 0.6 m wide and 0.5 m tall. The right post is tilted 15 degrees about x so the opening isn't a perfect rectangle. Behind the window there is a small shelf with a ball on it. In query 1 the arm starts rotated off to the left of the robot (hand near $(0.07, 0.55, 0.37)$) and has to reach through the window to $(0.56, 0.08, 0.44)$. In query 2 it starts off to the right (hand near $(0.20, -0.45, 0.38)$) and reaches higher through the window to $(0.62, 0.08, 0.50)$. The goals have about 4–5 cm clearance. The direct edge hits the left post in query 1 and the tilted right post in query 2.

The scenes therefore constrain different motions: the wall interrupts a lateral sweep while leaving routes around or above it; the frame obstructs approaches from either side to goals inside its opening. The window is an approach-alignment problem, rather than a rotated copy of the wall. Neither scene forces a unique route.



(a) wall query 1, start and goal

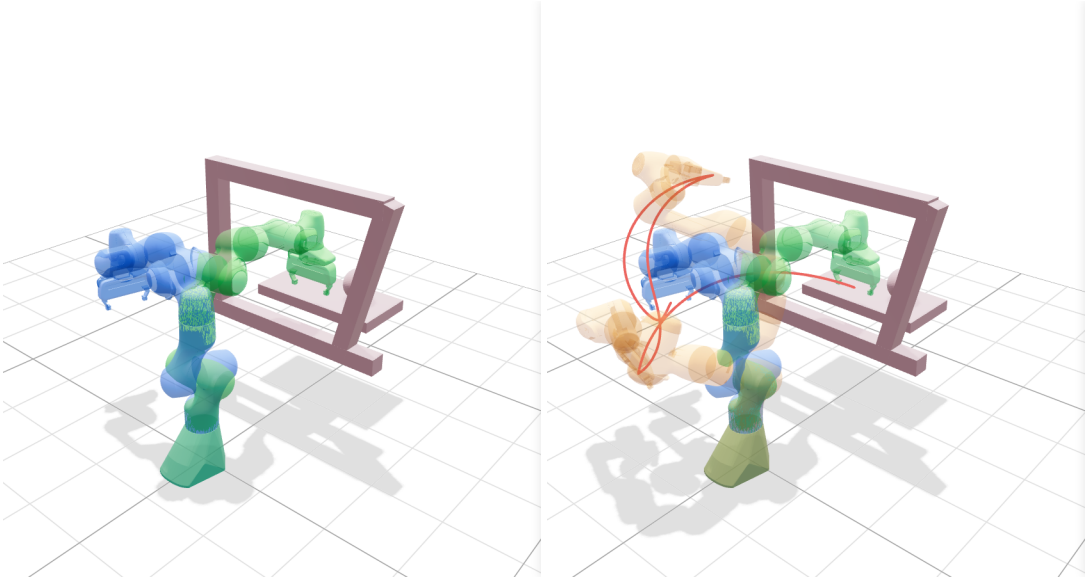
(b) wall query 1, RTP path (seed 1)



(c) wall query 2, start and goal

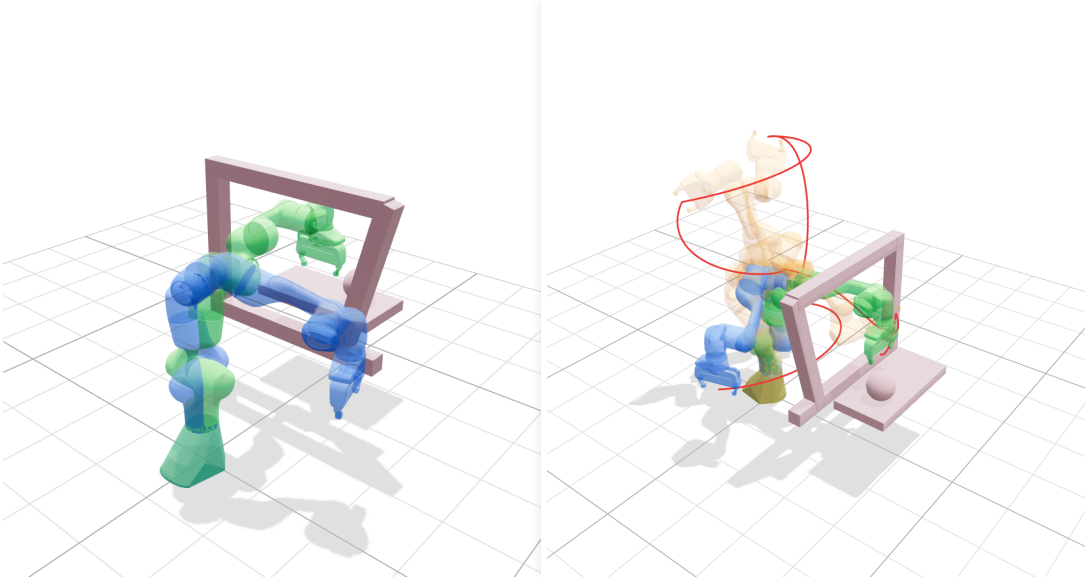
(d) wall query 2, RTP path (seed 5)

Figure 1: Wall scene in Viser. Blue: start; green: goal; translucent orange: intermediate waypoints; red: gripper trajectory. Wider path views include the full gripper trajectories.



(a) window query 1, start and goal

(b) window query 1, RTP path (seed 1)



(c) window query 2, start and goal

(d) window query 2, RTP path (seed 1)

Figure 2: Window scene in Viser. Colors match Figure 1; both saved paths are exact. The frame constrains the approach to the goals.

RTP paths and performance

The examples in Figures 1 and 2 were saved using `plan_arm.py` with goal bias 0.05, a 10 s limit, and collision resolution 0.005. The table reports individual saved runs, not repeated-trial medians. Per-trial records for repeated custom-scene experiments are unavailable, so we make no aggregate success-rate claim.

saved query	seed	time (s)	length (rad)	waypoints	tree size
wall 1	1	0.676	30.77	8	63
wall 2	5	0.201	9.61	4	19
window 1	1	0.117	11.80	4	12
window 2	1	0.603	24.46	7	55

The direct edge is blocked for all four queries. The wall leaves open space around and above it; the window constrains the arm's approach to the goal. RTP attempts full connections from uniformly selected vertices, so many sampled motions can intersect the frame. The saved window query 2 run with seed 2 is approximate after 10.065 s: its tree has 829 vertices, but the closest vertex to the goal is still the start. Returning that one-state path is consistent with the required approximate-solution rule. One failure and one success do not establish a success rate.

The exact examples have only 4–8 waypoints but make large swings and gripper loops. Their lengths are 9.61–30.77 rad, compared with straight-line distances of 1.20–1.63 rad. Those distances are lower bounds, since the direct connections collide; they are not achievable path lengths. The minimum clearance recorded at the planning resolution is 0.49–3.75 cm. Denser checking remains positive for each saved RTP path; for window query 2 it reduces the estimate from 1.40 to 1.22 cm. These observations show why short waypoint lists do not imply short or comfortably separated arm motions.

Original examples were produced on Linux. Fresh Linux and macOS smoke checks were evaluated separately and did not replace these measurements. The macOS build needed an Apple-only symbol-visibility correction; RTP source was unchanged. Seeded checks use fresh processes because resetting OMPL's global seed inside one process does not establish reproducibility. The main implementation challenges were state ownership, preserving solution paths after `clear()`, and selecting the correct approximate endpoint.

Contributions

Darshon Singh implemented and debugged RTP, managed state ownership, and led algorithm testing and planner analysis. Winson Lin developed the custom environments and start–goal queries, produced the Viser figures and saved paths, verified the results, and revised the documentation and report. Both contributed to testing, interpretation, report preparation, and submission preparation.

Exercise 1: difficulty 5/10, approximately 6 hours. Exercise 2: difficulty 6/10, approximately 7 hours. The main challenges were correct state ownership and choosing obstructed but solvable arm queries.